CHAPTER 21: TEAM ENABLEMENT AND THE DEBUGGING LOOP

Summary: *This domain is small on the blueprint and dense in detail, and almost all of it reduces to two questions. Which layer does this setting belong at, given who is supposed to be able to change it? And what do you observe before you touch anything? Get those two right and the configuration answers fall out; get them wrong and you will either hand every engineer a control the organisation meant to keep, or destroy the evidence that would have told you what broke.*

Put Each Setting at the Layer That Matches Its Audience

Configuration questions are layer questions. The right answer is nearly always the layer seen by exactly the **people meant to see and change** this, and the wrong answers put it one layer too private or one layer too public.

Two placements recur and both are worth memorising as a pair:

Organisation-wide enforcement goes above where any individual can reach. If the point is that nobody may turn it off, it has to live somewhere nobody can write to.

Team-shared configuration goes inside the versioned repository. If the point is that everyone working on this codebase gets the same thing, it belongs alongside the code itself, travelling on the same clone and the same pull everybody already runs.

That second one carries a general principle worth stating on its own. Anything **kept beside the code** travels on **machinery** that is **already proven to reach everybody**. Anything maintained per machine or per user diverges immediately, and nothing tells you it has.

The precedence stack is documented and worth knowing in order, highest first: **managed settings**, **command line arguments**, **project local settings**, **shared project settings**, **user settings**.[1] And a detail that catches people out: **installing Claude Code creates no settings file at all**.

Non-Overridability Is Not Reach

This distinction decides several items on its own, and the two properties are easy to conflate because both sound like “applies to everybody”.

Reach is how many people get the setting.

Non-overridability is whether any of them can turn it off.

A mechanism that **distributes a setting to every engineer** easily has **solved reach** and said **nothing about override**. When a scenario’s requirement is that something cannot be disabled, the only qualifying answer is a scope placed higher than anything a single person is able to edit. Convenience of distribution is a different property and it is the trap.

The managed-settings mechanics support this precisely. **Four managed settings sources** are checked in **priority order**, and the **first that delivers at least one policy key** is applied. And crucially: **array settings** such as the **permission allow and deny lists merge entries from all sources**, so developers can **extend** managed lists but **cannot remove** from them.[2]

Extend but not remove is the whole governance property in three words.

The same merging behaviour applies more widely: **list keys** such as the **permission allow list merge across files** rather than **overriding**.[1] A design assuming a local file replaces a list will find it added to one instead.

One trust-related asymmetry belongs here because it is a live security fact. From a committed project settings file, **allow rules, additional directories, declared marketplaces** and most environment values apply only **after each teammate trusts the folder**. **Deny and ask rules apply immediately**.[1] Permissive settings wait for trust; restrictive ones do not. That is the correct direction, and it means a **security control** shipped in a repository **works before anybody opts in**, while a **convenience** shipped the same way **does not**.

Memory Is Context. Hooks Are Enforcement.

If one distinction in this chapter is worth carrying into every scenario, it is this one.

Memory files are **loaded at the start of every conversation** and treated as **context**, not as **enforced configuration**.[3] They inform. They do not bind. A rule written into a project memory file is a strongly-placed suggestion, and the documentation is explicit about the remedy: to **block an action** regardless of what Claude decides, use a **pre-tool-use hook**.

Hooks are **user-defined shell commands** run at **fixed points in the lifecycle**, described as giving deterministic control so that **certain actions always happen** rather than **relying on the model to choose** to run them.[4]

That pairing answers a whole family of items. Whenever a scenario needs a guarantee, the answer runs **outside the model** altogether: a **program hanging off the event**, **permitted to refuse** what it sees. Every alternative depends on timing, on memory, or on persuasion, and none of those is a guarantee.

The **named anti-pattern** is the **convention written into a memory file** and **treated as a control**. It is the unencoded policy from chapter 15, in the configuration layer.

What Arrives Without Anyone Doing Anything

A recurring requirement: the applicable conventions must reach an engineer **without anyone taking an action**. No setup step, no command to run, no documentation to read first.

That rules out anything requiring a manual step at the start of a session, because a step repeated by hand every time is a step that will be skipped. The qualifying answer **attaches the instructions** to a place in the **directory tree** and **pulls them in from source control** on its own, so standing in that folder is the whole trigger.

Skills fit the same pattern with a useful economic property: a skill's body **loads only when it is used**, so **long reference material costs almost nothing** until it is needed. They are distributed three documented ways: **committed to a repository**, **shipped inside a plugin**, or **deployed organisation-wide** through **managed settings**.[5] Those three map exactly onto the three audiences: this project, this toolset, this organisation.

Subagents Pay Only When Recurring and Specialised

Defining a subagent earns its keep only when the work **comes back repeatedly** and needs **handling that differs from the default**. Both conditions, not one.

The reason is economic. What the definition buys you is a narrow prompt, a cut-down set of tools, and a model picked on purpose. All three are worth writing down only if they get **used again and again across sessions**, and worth having only if they actually **look different from what you would get anyway**. A definition written once for a single request never repays its cost, and a definition whose every distinguishing setting is left at the generic default adds a layer and changes nothing.

Versions That Move Under You

Two documented behaviours produce a specific class of “it worked yesterday” scenario.

A **plugin dependency tracks the latest available version by default**, so an upstream release can change what a plugin depends on with no warning. The documented remedy is a **semantic version range**, holding the dependency at a **tested version** until the **dependent team chooses to move**.[6]

Update checks run in the **background after a session starts**, with a **random delay of up to ten minutes**, and the running session **keeps whatever versions it loaded at launch** until **plugins are reloaded** or the **next session begins**.[7]

Put those together and you get the diagnosis for a whole family of confusing reports: two engineers on the **same commit** can be running **different plugin versions**, and one engineer’s session can **behave differently after ten minutes** than it did at launch. Neither is a bug and neither is visible in the repository.

Two Opposing Forces Make Scaling Safe

When a scenario asks how to scale tooling across teams safely and wants two answers, the pair is always **opposing**.

One force propagates a vetted default outward, so teams inherit something known-good instead of inventing their own.

One force **caps what automation may do** when that **default turns out to be wrong**. A limit on blast radius, independent of the default's quality.

Both are required precisely because the first one is fallible. And the reason there is never a defensible third pick is that the remaining options **remove a control rather than add one**: broadening a permission, relaxing a restriction, granting a capability more widely. **Scaling is not the same as loosening**, though it is very often proposed as though it were.

The same logic settles the permission-narrowing scenario. When a grant is too broad, the correct move **cuts back what that grant allows** and leaves the **thing enforcing it switched** on. Both halves. Widening the surface fails obviously; removing the mechanism that would constrain it fails less obviously and is worse, because it takes the control away in the name of fixing it.

Two Constraints on Machine Output That Reaches People

Where generated material goes somewhere consequential, the workable design satisfies two things at once:

Whatever it asserts is **pulled from the real source** at the **moment it is written**. Not from a snapshot, not from recall.

Nothing reaches the outside world until a **person has passed** it.

Every wrong option drops one of the two, and dropping either is tempting: a cached source runs faster, and skipping the person runs smoother. **Neither compromise survives** contact with the **first wrong output that reaches a customer**.

A related property decides the review scenarios: what makes a review meaningful is that whatever does the **reviewing** knows **nothing of what did the writing**. Not how much review effort is applied, and not how narrowly the assistant is allowed to write. The **same context that produced the work cannot independently check** it, however carefully it is asked to.

Automated Review Adds Analysis Without Moving the Merge Gate

The same principle has a documented instance in the pull request workflow, and it is worth pinning down because the scenarios reach for it constantly. A team wants machine help on review and wants the decision to stay with a person. Those two wishes are usually presented as a trade, and they are not one.

What the documented behaviour actually does: several agents examine the diff and the code around it, a verification step checks each candidate finding against how the code really behaves so that false positives get filtered out, and the surviving findings are ranked and **posted as comments on the lines they concern**, with a summary alongside them.[8] The output is an **input to the human decision**. Nothing in it touches branch protection, so whatever approval the repository already required is still required.

That is the shape to recognise, and the wrong options are built by breaking it in one of three places. **Granting the reviewer merge rights** removes the gate. **Turning off branch protection** removes the mechanism that enforces the gate. **Recording a machine approval against the reviewer requirement** keeps the gate and empties it, which is the subtlest of the three and the one most likely to be argued for on the grounds that a check ran.

The **named anti-pattern is the satisfied gate**: a control that still exists on paper, still shows green, and no longer has a person behind it. It is the uninformed approval from chapter 18 with the person removed rather than uninformed, and it fails the same test. Ask what the gate would do differently if the analysis were wrong.

Measuring Whether Any of This Helped

Adoption metrics are where enablement programmes go to feel successful, and the exam is precise about what counts.

Two things together make a signal mean anything: a **result compared against how things stood before**, and a **described mechanism** explaining where the change came from. The number tells you something changed; the account tells you whether the change is attributable to the thing you did.

Everything else measures **activity volume** or **unverifiable self-report**. Sessions started, messages sent, engineers who say it helps: none of these is a delivery outcome, and all of them go up when a tool is merely interesting.

One documented metric deserves specific care because it is so easy to over-read. **Lines of code accepted** counts the lines Claude Code **wrote that users accepted** in their sessions. It **excludes rejected suggestions and does not track subsequent deletions**. More importantly, the contribution metrics are **deliberately conservative and an underestimate**, counting only cases of high confidence in the tool's involvement, and code that developers **substantially rewrote**, defined as **more than 20 percent different**, is **not attributed at all**.[9]

Two consequences. The number **understates** the tool's involvement, so treating it as an upper bound is wrong. And it is an **input measure**, not an outcome: accepted lines are not shipped value, and a workflow improvement that results in *less* code is invisible to it.

Observe Before You Change

The debugging half has one organising principle and it survives every scenario.

Take the **cheapest observation** that **separates the leading hypotheses**, while leaving the **state you are diagnosing intact**.

Name the discipline: the **verification checklist** is the short run of observations that pins down the faulty layer before anybody touches a setting. Its purpose is not caution for its own sake. It is that a **change made before diagnosis destroys the attribution**: if you alter two things and the symptom clears, you have lost which one mattered, and you may have masked a cause you never addressed.

Three rules follow, and each maps to a family of items.

Where the evidence already points, go and **check that it is right before you act** on it. The next move is whatever observation would confirm the reading, not the expensive or irreversible step that assumes it. This is the difference between a diagnosis and a bet.

Prefer moves that **produce evidence** while **leaving the thing you are investigating alone**. When a scenario asks for two next steps, the pair is always the two that only

look. The rejected three **perturb the system**, **discard the state**, or **hand the problem to somebody else**, and each of those destroys or forfeits the evidence.

Work out **which layer is at fault** before **touching a single setting**. That is what keeps the eventual fix attributable to anything.

The sharpest instance is the access-denied scenario. Something is blocked, and the tempting response is to widen access until it works. That **destroys the very information** that would tell you whether this was an **intentional policy block** or a **misconfigured rule**, which are two completely different problems with two completely different fixes. Diagnose first; the symptom is identical and the causes are not.

And when the problem has to be escalated to somebody else: give them whatever **distinguishes the runs that fail** from the runs that work, and ask for **candidate explanations** ordered by **how cheaply each can be tested**. That turns the exchange into an experiment rather than a request for speculation, which is what an escalation usually degenerates into.

What Actually Loaded

Most configuration mysteries resolve the moment somebody looks at what the session actually loaded, and there are documented commands for exactly that.[10]

To find out	There is
What context is in play	A breakdown listing memory files, skills, and the source each custom subagent loaded from
Which settings are active	A status view naming the settings sources, including which managed source applies
Which hooks exist	A hooks listing

To find out	There is
Whether a server connected	A server listing with connection status and per-project approval
Whether a customisation is the cause	A safe mode that launches with customisations disabled
Why a server returns no tools	A debug flag, with the error output read from the session's debug log

Two documented **silent** failures are worth memorising, because neither produces an error and both waste a day:

A **misspelled tool name in a hook matcher matches nothing**. The hook exists, the configuration is valid, and it never fires.

A **project-scoped server stays disabled** because its **one-time approval prompt was dismissed**. The configuration is correct and the server is simply off, with nothing on screen to say so.

The Headless Trust Gap

One documented behaviour is a genuine security finding and belongs in every review of automated runs.

A **non-interactive run** loads the **same context an interactive session would**, unless **bare mode** is used. Bare mode skips auto-discovery of hooks, skills, custom commands, subagents, plugins, servers and memory files, and is recommended for continuous integration and for scripts that must produce the same result on every machine, precisely because bare mode **never reads a hook** sitting in somebody else's **home directory**, nor a **server declared in the project's own configuration**, so **neither one runs**.[11]

Now the part to sit up for. Without **bare mode**, a **non-interactive session** runs the **hooks** in a project's settings file and **connects the servers** it declares even in a folder that has **never been trusted**, showing **no workspace trust dialog** and **no per-server approval prompt**.[11]

The interactive trust gate is a human prompt. **Remove the human and you have removed the gate**. Any pipeline that checks out untrusted code and runs a non-interactive session without bare mode is executing whatever that repository declared.

How Wrong Answers Are Built on Enablement and Debugging Questions

- **Reach offered where non-overridability was required**. The test: can an **individual turn this off**?
- A **memory file used as enforcement**. The test: what **happens if the model ignores** it?
- A **manual step where automatic arrival was required**. The test: does **anybody have to do anything** for this to apply?
- A **control removed in the name of scaling**. The test: does this **add a constraint** or **relax one**?
- A **change made before an observation**. The test: after this action, can you still tell which **hypothesis was right**?
- **Activity counted as outcome**. The test: is this a **delivery result measured against a baseline**?

When the scenario says	The answer is usually	The trap is usually
A setting must not be disableable by anyone	A scope higher than anything a single person can edit	An easy way to distribute it to everyone, which is reach, not override
A team must share configuration	Put it beside the code, travelling on the same clone and pull	Per-machine or per-user setup, which diverges silently

When the scenario says	The answer is usually	The trap is usually
A convention must apply automatically in a directory	Attach it to that place in the tree and pull it from source control	Anything needing a step at the start of each session
A rule must hold regardless of what the model decides	A pre-tool-use hook, outside the model, permitted to refuse	A memory file, which is context and not enforcement
Developers should not be able to remove an organisational entry	Managed list settings merge, so entries can be extended and not removed	Assuming a local file overrides the list
A security control ships in a repository	Deny and ask rules apply immediately, before anyone trusts the folder	Assuming everything waits for trust
A subagent is proposed	Worth it only when the work recurs and needs non-default handling	A one-off task, or a definition left at generic defaults
Tooling must scale safely, two answers wanted	Propagate a vetted default, and cap what automation may do when it is wrong	Anything that broadens a permission or relaxes a restriction
A permission grant is too broad	Cut back what it allows, leaving the enforcing mechanism switched on	Removing the enforcement mechanism

When the scenario says	The answer is usually	The trap is usually
Generated material will reach customers	Facts pulled from the real source as it is written, and a person passing it before publication	Cached sources, or automatic publication
A review must be meaningful	Whatever reviews must know nothing of what wrote	More review effort, or narrowing what may be written
Adoption must be measured, two wanted	A result against how things stood before, plus a described mechanism	Session counts, message volume, or survey self-report
Lines of code accepted is quoted	An underestimate, an input measure, ignoring rewrites over 20 percent different	Reading it as shipped value or as an upper bound
Two engineers on the same commit behave differently	Plugin dependencies track latest by default; pin a version range	Looking for the difference in the repository
A session changes behaviour partway through	Background update checks run with a random delay of up to ten minutes	Assuming versions are fixed for the session
Access is denied and the work is blocked	Diagnose whether it is intentional policy or misconfiguration first	Widening access, which destroys the distinction

When the scenario says	The answer is usually	The trap is usually
Two next steps are wanted in a live incident	The two that only look, leaving the system alone	Anything that perturbs, discards state, or hands it on
Something must be escalated	Whatever distinguishes failing runs from working ones, and candidates ordered by test cost	A description of the symptom and a request for ideas
A hook never fires and nothing errors	Check the matcher for a misspelled tool name	Assuming the hook is running and failing
A project server is not connected	Its one-time approval prompt may have been dismissed	Debugging the server itself
A pipeline runs Claude Code non-interactively	Use bare mode; otherwise hooks and servers from untrusted code run with no prompt	Assuming the trust gate applies without a human
Machine help is wanted on review without losing human approval	Findings posted as comments on the lines concerned, with the merge gate untouched	Granting merge rights, disabling branch protection, or recording a machine approval against the requirement
User scope versus other scopes, two wanted	Preferences belonging to one person that should follow them into every project	An organisation-wide policy, a server definition shared by one repository, or credential storage

When the scenario says	The answer is usually	The trap is usually
Engineers re-explain the same project context in every session	A project context file committed to the repository and picked up when a session opens	Performing the same manual step faster, or deleting the conventions so there is nothing to explain

The habit worth carrying: **before changing a configuration, look at what actually loaded**. Most of these scenarios are somebody debugging a file that was never read.

What to Remember

- Put each setting at the **layer seen by exactly the people meant to change** it. **Organisation-wide enforcement sits above where any individual can reach**; **team-shared configuration sits beside the code**, travelling on the same clone and pull everybody already runs. Per-machine and per-user copies **diverge silently**.
- Precedence, highest first: **managed settings**, **command line arguments**, **project local settings**, **shared project settings**, **user settings**. Installing Claude Code **creates no settings file**.
- **Reach and non-overridability** are different properties. Easy distribution to every engineer says nothing about **whether any of them can turn it off**. Only a scope placed **higher than anything a single person can edit** satisfies “cannot be disabled”.
- **Four managed settings sources** are checked in **priority order** and the **first delivering at least one policy key applies**. **Array settings merge across sources**, so developers can **extend managed lists but not remove** from them. List keys such as the allow list **merge rather than override**.
- From a committed project file, **allow rules, additional directories, marketplaces** and most environment values apply only **after a teammate trusts the folder**; **deny and ask rules apply immediately**. A restriction shipped in a repository **works before anyone opts** in.

- Memory files are **context, not enforced configuration**. To block an action regardless of what the model decides, use a **pre-tool-use hook**, which runs **outside the model** so **certain actions always happen** rather than depending on the model choosing to run them.

- A requirement that conventions arrive with **nobody doing anything** is met by **attaching them to a place in the directory tree** and **pulling them from source control automatically**. **Any manual step at session start** will be skipped.

- A skill's body **loads only when used**, so long reference material is **nearly free until needed**, and skills are distributed by **committing to a repository**, **shipping in a plugin**, or **deploying through managed settings**.

- Defining a subagent earns its keep only when the work **comes back repeatedly** and needs **handling that differs from the default**, because a narrow prompt, a cut-down toolset and a deliberately chosen model only **repay what they cost** once they are **used again and again**, and a definition **left looking like the defaults** adds nothing at all.

- **Plugin dependencies track the latest version by default**, so an upstream release **changes them without warning**; a **semantic version range** holds them at a tested version. Update checks run in the **background after session start** with a **random delay of up to ten minutes**, and a running session **keeps the versions it loaded** until a reload or the next session.

- Scaling safely needs **two opposing forces**: **propagate a vetted default**, and **cap what automation may do** when that **default is wrong**. There is **no defensible third choice**, because the remaining options **remove a control rather than add one**. Scaling is **not loosening**.

- A too-broad grant gets **cut back with the enforcing mechanism left switched** on. **Removing the mechanism** is worse than the original problem.

- Machine output reaching people needs **both**: whatever it asserts **pulled from the real source** at the **moment of writing**, and **nothing published until a person has passed** it. And a review means something only when

whatever reviews knows nothing of what wrote, not through more effort or a narrower writing scope.

- Adoption signal means something only as a **result compared against how things stood before**, alongside a **described mechanism for the change**. Session counts, message volume and self-report **measure activity**.
- Lines of code accepted **excludes rejections**, does **not track later deletions**, is **deliberately conservative**, and **ignores code rewritten by more than 20 percent**. It **understates involvement** and is an **input measure**, so a change that produces less code **looks like a failure**.
- Take the **cheapest observation that separates the leading hypotheses** while **leaving the state intact**. A **change made before diagnosis destroys attribution** and can **mask a cause it never addressed**.
- **Check that the reading is right before acting** on it, prefer moves that **produce evidence while leaving the system alone**, and **work out which layer is at fault** before touching a single setting.
- When access is denied, **diagnose before widening** it, because widening **destroys the distinction** between an **intentional policy block** and a **misconfigured rule**.
- When escalating, give them **whatever distinguishes failing runs from working ones** and ask for **candidate explanations** ordered by **how cheaply each can be tested**. That makes it an **experiment** rather than a **request for speculation**.
- **Two documented silent failures**: a **misspelled tool name in a hook matcher matches nothing**, and a **project-scoped server stays disabled** because its **one-time approval prompt was dismissed**. Neither **produces an error**.
- A non-interactive run **loads the same context as an interactive one** unless **bare mode** is used, which skips auto-discovery of hooks, skills, commands, subagents, plugins, servers and memory files and is recommended for CI.

Without bare mode, a non-interactive session **runs a project's hooks and connects its declared servers** even in a **folder nobody has trusted**, with **no trust dialog and no per-server prompt**. **Remove the human** and you **remove the gate**.